UNIVERSITY OF LEEDS

Gen AI

AI tools can be used in an assistive role

School of Computing: Assessment brief

Module title	Software Engineering
Module code	COMP2850
Assignment title	Group Project
Assignment type and description	In this assignment you will work on a given project with a group of your peers to produce a web application. You will be responsible for developing the requirements, designing, and evaluating your user interface and user experience, and incorporating user feedback to iteratively develop a system.
Rationale	As a software engineer, you need to be able to work across all aspects of the project, not only writing the code but also understanding your users and their needs, producing suitable documentation, and developing applications which meet those needs.
Word limit and guidance	You should spend around 8 hours per week working on your project – the time that you would spend in lectures and in individual study. Individual reflection: 2000 words.
Weighting	100%
Submission deadline	17:00 on Friday 8th May Demonstrations will be held on Monday 11th May
Submission method	Code, Wiki and Board: GitHub Individual Reflection: Gradescope Weekly Diary: Gradescope Check-Ins: Feedback Fruits Poster: Gradescope Demo: In-person

Feedback provision	You will have the opportunity to receive feedback from your product owner (a member of module staff) throughout the project on request. Group feedback via email. Individual feedback via Gradescope.
Learning outcomes assessed	1. apply subject specific knowledge and engineering design principles to design and implement software artefacts which satisfy complex real-world requirements, considering accessibility and inclusive design principles. 2. select and interpret sources of information to solve complex real-world problems. 3. use appropriately selected tools and processes to design, test, analyse and evaluate computer systems and identify limitations. 4. identify and analyse ethical and sustainability concerns when designing and implementing software and make reasoned decisions informed by ethical frameworks and codes of conduct to minimise adverse impacts. 5. identify and interpret risk assessing the potential impact and plan mitigation approaches. 6. apply and discuss the principles of quality management in software engineering and systems design. 7. Operate effectively as a member of a team in various software engineering roles. 8. apply and discuss engineering management principles demonstrating awareness of commercial context, project and change management, and relevant legal matters. 9. communicate effectively complex topics related to programming and software engineering to technical and non-technical audiences. 10. reflect on their level of mastery of subject knowledge and skills and plan for personal development.
Module lead	Amy Brereton – a.m.brereton@leeds.ac.uk

1 Assignment guidance

You will produce a web application based on one of several different system specifications.

The specification will provide a basic high-level overview of the system. You will use this to help plan a needs-finding discussion with a product owner, which you will then use to develop a comprehensive set of user stories which will be prioritised, and developed to produce a working webapp.

You will work as a self-organising team, producing and iteratively improving a system based on your initial needs-finding and from peer and product owner feedback. You will use different testing methods including unit testing, UX testing and end-to-end testing throughout development and use this to shape your development.

As a group, you will select and use appropriate tools to manage your work, develop your code and handle version control. You will also develop some minimal documentation, both of your system and of your design and testing.

You and your team will also be responsible for organising weekly team meetings and at least two retrospectives throughout your project – minutes and actions will be taken for these meetings.

2 Assessment tasks

You will submit:

- A GitHub repository containing your codebase.
- A Wiki containing your documentation.
- An individual reflection.
- 5+ Weekly development diaries.

Throughout the project, you will complete:

- One in-person check-in with your product owner
- Two online check-ins

These will ensure that everyone is contributing and engaged, and any inter-team issues can be highlighted and resolved.

At the end of the project, you will produce a poster pitching your system and its features and benefits, and you will also produce a demonstration of your system which will show your features, building on your initial work producing personas and job stories.

This will be presented in person to a member of staff.

A breakdown of each deliverable is given below:

Codebase

Your final source code will be submitted. This should be working with no bugs on the main branch (you may have issues on development branches), well documented with file- and function-level comments used to ensure your code is maintainable and readable. Your code should be consistently styled across files and functions, with suitable variable, class, and function naming.

You must also test your code, and your tests will form a part of your assessed work.

Documentation

On your GitHub Repository you should have a Wiki containing your documentation including:

- meeting and retrospective notes
- personas
- job stories
- design and planning documents.
- database diagrams
- class diagrams

These will not be ‘formal’ documents but rather a way for you to keep track of your own planning and designs- you should not be producing these only to meet the specification, they will be developed by your team throughout your project to agree on certain standards and designs.

Your GitHub Project board will also form a part of your documentation – this should be used to both track user stories through the project and for issue and bug tracking.

You should keep older versions of these files as you adapt and modify them, with notes about any changes justifying why they were changed.

Individual Reflection

Each week there will be a **portfolio task** of completing a weekly development diary – this is a short set of reflective questions for you to answer on Gradescope and will help to evidence your work on the project.

The project runs over 7 weeks (week 4 – week 11) – you must complete a minimum of 5 weekly development diaries in this time in order to pass portfolio requirements.

To ensure that diaries are an accurate reflection of the week, there is a **strict time window** to complete the previous week's diary.

At the end of the project, you will complete a short (~2000 word) reflection on your personal growth throughout the project. A template will be provided for this reflection, and you will be able to use your weekly diaries to remind yourself of previous weeks.

Poster & Demo Session

At the end of the project, you will do a final presentation and demonstration of your work. You will be pitching your system using a poster and a demonstration and should be prepared to talk about what you did and parts of the system of which you are particularly proud.

You will demonstrate your system- this should be a practiced professional demonstration which is based on personas and job stories and demonstrates the full functionality of the system. This should take around 15 minutes.

This demonstration will be to 2 members of staff.

Check Ins

There are a total of 3 check-ins throughout the project. The first check-in is your in-person product owner meeting where you will meet with your product owner and have an opportunity to discuss your project and ask any questions needed for requirements gathering. Your attendance will be monitored for this meeting.

The other two check-ins will be online peer feedback check-ins completed through Feedback Fruits; you will be asked to give a rating for each of your team members (and yourself) based on:

- Contribution
- Communication
- Quality of Work

This will be an opportunity to reflect on the time since the last check-in, highlight any team issues to us, and to recognise any team members who are contributing more than others.

Team Allocation

You will work in a team of 4, and teams are assigned using a survey tool which will match you based on several criteria:

- Students who want to do the same projects (priority 1)
- Students who have a similar timetable (priority 2)
- Students with different working styles (priority 3)

This is designed to be the fairest way of creating groups, ensuring that you can work on the project you want to, that you have time to do so, and that you are working with a range of different styles in your group.

You cannot change teams, and you will need to work with people who you may not know – this is part of working life, and it's a useful skill to learn how to get along with people who may have different interests, personalities and styles than yourself. If you struggle with groupwork and find yourself needing additional support, please reach out.

In cases where there are serious disagreements, clashes, or unacceptable behaviour in a group, please email the module leader ASAP.

3 General guidance and study support

Team Working, Communication and Expectations

You will be working in a team, meaning that you need to ensure you are communicating with your group and that everyone knows what everyone else is doing. You should have regular (weekly) meetings and should have some more frequent informal communication about your work.

Meetings are a chance to discuss any ongoing issues, make decisions about design or changes, and make sure everyone is up to speed with where the system is.

You will have a retrospective after each sprint which will be a way of changing working practices, resolving any issues or conflicts within the group, and incrementally improving how you work as a team.

If you have serious conflicts within the group, or concerns about someone's work, please raise them in your check-ins or by messaging module staff. Where the concerns are minor (e.g. someone's code is not as high standard as expected) then please discuss this in group and handle this in a professional way such as pair-programming with the team member and supporting them to improve.

If someone does not engage with the work, does not communicate with the team, etc. please raise this as early as possible to module staff. If that person does later engage with the group, please try and include them- make sure they are assigned some work to do and try and integrate them into the group.

Individual Working

At times, you will work alone on parts of the system or on code. You should ensure you are doing this to the standards set by your group, following expected conventions, and not making large changes to other peoples' code without discussion.

You should track what you are working on using your project board to ensure everyone knows what everyone else is doing.

Use of Git

You will be using Git for version control throughout the module. You should, as a team, decide on a strategy for how you will work – for example, whether you will use feature branches or developer branches, and how pull requests on main will be handled.

You should **not** program directly on the main branch – your main branch is for finished, tested, working code only. Code should be developed on branches and pull requests used to merge into Main only when tests have been passed, and code is confirmed to be working correctly.

Testing

You are expected to thoroughly test your code, both to ensure it does what you expect it to do, and to ensure that you have considered a range of potential security risks. You should try and make sure that there is no ‘unknown’ behaviour – if we consider the example of the Wordle portfolio task, this involves asking yourself:

- How does my program react if the words.txt file is empty?
- How does my program react if the words.txt file doesn’t exist?
- How does my program react to empty user inputs?
- What happens if a word contains 4 letters, or numbers/special characters?

Consider what may create exceptions in your program, how this should be handled gracefully, and what you can do to prevent your user receiving error messages. Remember – you’re writing these programs for a customer.

You should also plan this with your team – think about how your classes and functions interact, and where the best place to put validation will be and therefore where the tests should be.

It is generally recommended that the person who writes the code should not be the person who writes the tests. We also recommend trying test-driven development (writing the tests first).

You may do some manual testing, for example testing full user journeys – in these cases, consider how you document these, when they should be done, and how the feedback is given to the developers.

4 Assessment criteria and marking process.

You will be marked based on:

- Documentation [20]
- Source code [20]
- Final produced system [30]
- Individual Reflection [10]
- Checks Ins [10]
- Poster Presentation and Demo [10]

Full marking rubrics provided in Section 8.

5 Presentation and referencing

Code

You should follow basic standards for your code - Kotlin code conventions.

You may wish to use KDoc for richer in-code documentation.

Tools such as Detekt and KLint will support you in finding and fixing any standardisation errors.

Writing

Your reflection is a relatively informal piece of academic writing, and you can use the first person (I, me, my) if you would like to. The Library has a guide to reflective writing which you may find useful – you are expected to fully reflect and not simply give an account of what you did.

Any references should be Leeds Harvard style – your in-text citation should include the author's surname(s) and the year of publication, with a corresponding item in the bibliography.

You can find a full list of the correct bibliography styles for different types of media, including websites, journal articles and books through the link above.

6 Submission requirements

Codebase, Documentation & GitHub

Make all final commits and pushes **before** the deadline, as you will need to submit your final repository by the deadline.

Weekly Diaries and Reflection

Weekly diaries will be submitted each week through Gradescope. The diary will open for submission on Friday morning and will close the following Thursday afternoon. This gives you a week to write each previous week's diary.

Individual reflections will be submitted via Gradescope at the end of the project.

Check-Ins

You will have a 2-week window to complete your online check-ins. These will be completed in Feedback Fruits, through Minerva.

Poster Session & Demo

You will submit a PDF of your poster, and you will be required to attend the poster session and demonstrate your system to staff on the day.

7 Academic misconduct and plagiarism

Plagiarism and Academic Integrity

Leeds students are part of an academic community that shares ideas and develops new ones.

You need to learn how to work with others, how to interpret and present other people's ideas, and how to produce your own independent academic work. It is essential that you can distinguish between other people's work and your own and correctly acknowledge other people's work.

All students new to the University are expected to complete an online Academic Integrity tutorial and test, and all Leeds students should ensure that they are aware of the principles of Academic integrity.

When you submit work for assessment it is expected that it will meet the University's academic integrity standards.

If you do not understand what these standards are, or how they apply to your work, then please ask the module teaching staff for further guidance.

By submitting this assignment, you are confirming that the work is a true expression of your own work and ideas and that you have given credit to others where their work has contributed to yours.

Generative AI

This assignment has been rated **amber** for use of generative AI. This means that you may use generative AI tools in a supportive manner.

You can use generative AI to:

- Proofread and translate text.
- Generate spoof data for testing.
- Summarise and explain parts of the specification for your own clarity.
- Support you with debugging (suggesting where you may have made a mistake in your code, or deciphering an error message)
- Supporting you with understanding a concept or piece of code which you are struggling with.
- Support you in developing front-end code

You can use generative AI to support you with parts of the code which we don't expect you to be very familiar with such as front-end development; you should still test and ensure you understand this code and **must** acknowledge where generative AI has been used. You should add a comment stating which lines were written primarily by generative AI and which model was used.

```
# used Copilot to explain how to edit background colour (line 17-21)
```

8 Assessment/ marking criteria grid.

Codebase, Wiki and System [70 marks]

Criteria	Fail (< 40%)	Pass (40%)	Good (55%+)	Excellent (70%+)
Documentation [20]				
Requirements Gathering and Needs-Finding **[8]**	No or insufficient evidence of requirements gathering. Personas not produced or too basic for purpose. Very limited evidence of job stories or user stories.	Clear evidence of requirements gathering. Multiple well-defined personas which are clearly linked to developed job stories, and user stories which have been estimated and prioritised.	Personas and job stories encapsulate a wide range of users. User stories created which capture full system and some additional features. Acceptance criteria which are linked to tests.	Personas and job stories have clear impact on all elements of the system. Evidence of how priorities and user stories have developed throughout project with justification. Testing strategy centred on personas and stories.
UX and UI Design **[7]**	No or basic wireframes for site. No evidence of incremental change. Designs backported from final system.	Annotated wireframes encapsulating key design choices such as colours and icons. Evidence of incremental changes to design. At least one UX test performed with feedback used to improve.	Incremental changes to designs have been well-reasoned and have evidence chains. Evidence of multiple UX tests.	Full and deep justification of all significant design changes made. UX testing integrated throughout project and clear link to improvements and refactoring.

Meeting Notes [2]	No or extremely basic notes from meetings. One person produced all minutes.	Concise but informative meeting notes which list discussion points and actions. Retrospectives recorded with clear outcomes. All team members contribute to minuting.	Good action tracking, with assigning and monitoring of previous actions and outcomes. Outcomes of retrospectives feed into future retrospectives.	
Use of Git & GitHub [3]	Poor evidence of Git usage. Code on main that does not work. No evidence of a strategy.	Reasonable use of Git, with branches used in a sensible way to prevent non-working code on main.	Consistent strong use of Git with a clear strategy. Issues tracked and assigned to users.	Excellent use of Git with a clear strategy including Actions or other CICD tools.

Source Code [20]				
Coding Standards [10]	Inconsistent standards / no clear standard followed. Linter highlights multiple significant issues.	Code follows the expected Kotlin standards. Linter may highlight one or two significant issues. File and function level comments.	Linter highlights only few minor issues. All comments follow some consistent format.	No linter issues. Comments use KDoc or similar format throughout. Code is extremely consistent. Team have set own standards which have been followed by all developers.
Testing [10]	No tests, or very few tests implemented. No automated testing. Not clear that tests were used in development.	Good range of automated tests covering different functionality in the webapp. Evidence of tests being run throughout.	Excellent test coverage, with a wide range of unit tests and integration tests. Tests used frequently and used to assist development.	Use of CICD tools to integrate testing into Actions, near-perfect test coverage and well-designed integration tests. Evidence of security testing.

Final functional system [30]				
Functionality [20]	Extremely basic system which meets some but not all requirements. Errors in final system.	All basic requirements met with some considerations of security. Errors handled gracefully with no crashes.	Some additional work outside of basic scope has been included. An additional feature which has been added. May be partial or not fully integrated. Good consideration of security and validation.	A high-quality additional feature or extension has been completed. Extension has a clear use case and is integrated well into the basic system. Thorough and extensive consideration of security and validation throughout project.
UX [10]	Basic UI which is not substantially changed from a base library. WCAG errors or a clear lack of consideration of accessibility.	A suitable UI which has been somewhat personalised from a base library. No Level A WCAG errors, and limited AA. Accessibility has been considered for a variety of users.	Intuitive and professional UI with some significant changes from base library. No significant accessibility errors.	Accessible-first UI which has been designed and improved based on personas and job stories. Consistent wording, formatting, and attention to detail. No accessibility errors.

Individual Reflection [10 marks]

Criteria	Fail (< 40%)	Pass (40%)	Good (55%+)	Excellent (70%+)
Technical Concepts & Problem-Solving [4]	Very limited or no discussion of technical decisions. No evidence of problem-solving processes. Descriptions are surface-level or inaccurate.	Clear explanation of key technical choices made during the project. Some evidence of debugging or problem-solving steps. Shows basic understanding of programming concepts used.	Well-reasoned reflections on technical decisions with explicit links to project outcomes. Explains iterations, debugging strategies, and rationale for approaches taken.	Deep and insightful evaluation of technical decisions and their impact. Clear evidence of iterative problem-solving with examples. Strong justification of methods, alternatives considered, and lessons learned.
Team Working [3]	Minimal or unclear description of individual contribution. Little or no evidence of collaboration. Team dynamics not discussed or inaccurately described.	Clear description of personal responsibilities and contributions. Evidence of collaboration and communication. Basic reflection on team processes.	Strong reflection on individual contribution with examples. Insight into communication, coordination, or conflict resolution. Team processes linked to project outcomes.	Detailed, thoughtful evaluation of team dynamics. Shows awareness of how collaboration evolved over 7 weeks. Critical reflection on contribution, challenges, and improvements.

Depth of Reflection [3]	Work is purely descriptive with no meaningful reflection. Does not identify challenges or learning. No evidence of personal growth.	Basic reflection on successes and challenges. Some evidence of learning or development. Shows awareness of improvements needed.	Clear, thoughtful reflection that moves beyond description. Identifies turning points, challenges, and key lessons. Demonstrates self-awareness and growth.	Deep and analytical reflection. Insightful evaluation of decisions, challenges, and development. Shows critical thinking and connection to future practice.

Demo [10 marks]

Criteria	**Fail (< 40%)**	**Pass (40%)**	**Good (55%+)**	**Excellent (70%+)**
Technical Demonstration [5]	Demonstration incomplete, unclear, or non-functional. No connection to earlier requirements gathering (personas, job stories, user stories). Unprofessional delivery.	Working demonstration with clear explanation of core features. Some attempt to link features to user needs or user stories. Delivery mostly clear.	Strong, structured demonstration of key functionality. Features clearly linked to personas, needs-finding, or user stories. Professional delivery with confident explanations.	Highly polished and professional demonstration. Clear end-to-end narrative connecting features to personas, needs, priorities, and user stories from the early project stages. Demonstration highlights design decisions, reasoning, and impact on users.

Responses to Questions [2]	Unable to answer questions or answers incorrect. Shows limited understanding of the system.	Answers basic questions with reasonable accuracy. Shows general understanding of system features and decisions.	Confident answers demonstrating good understanding of technical choices and rationale. Able to reference earlier design/requirements work when appropriate.	Insightful, well-reasoned answers that show deep understanding of technical, design, and user-centred decisions. Draws explicitly on personas, stories, testing, and design evolution.
Poster [3]	Poster unclear, incomplete, or confusing. Fails to pitch the system or convey its purpose. Key features and uses not explained.	Poster presents the system clearly with a basic explanation of features and intended users. Overall message understandable. Layout is functional.	Strong, well-organised poster that communicates the system's purpose, core features, and key use cases. Pitch is coherent and persuasive. Visually professional.	Highly engaging and polished poster that effectively pitches the system. Clearly communicates the value, features, and real-world uses of the product. Strong visual hierarchy and messaging that appeals to target users or stakeholders.

Check-Ins [10 marks]

You will receive 0-5 marks per online check-in based on your contribution as judged by your teammates.

You will receive zero marks if you do not complete the check-ins.

These marks will be calculated based on team averages, and any outlier marks will be investigated to prevent unfair or malicious behaviour.